Event Planner

Project Report

April 25, 2026

Contents

1 Overview **2**

2 Data Model **2**

3 API Endpoints **2**

3.1 POST /api/events 2

3.2 POST /api/tickets 3

4 Challenges **4**

1 Overview

This is a full-stack event planning application built with Node.js / Express on the backend and React / TypeScript on the frontend. It allows users to create and manage events, register attendees, and issue tickets that link attendees to events. The backend exposes a REST API backed by MongoDB Atlas via Mongoose, deployed as a serverless function on Vercel. The frontend is a single-page application using React Router v7 for client-side routing, a global store via React Context, and no UI framework — all styles are handwritten CSS using custom properties. Data flows through typed service functions that call `apiFetch`, a thin wrapper around the browser `fetch` API.

2 Data Model

The application uses three MongoDB collections: `events`, `attendees`, and `tickets`. A ticket acts as a join document, holding a reference to both an event and an attendee, along with a type (`standard` or `vip`) and a status (`active` or `cancelled`). A unique compound index on `(event, attendee)` prevents duplicate registrations.

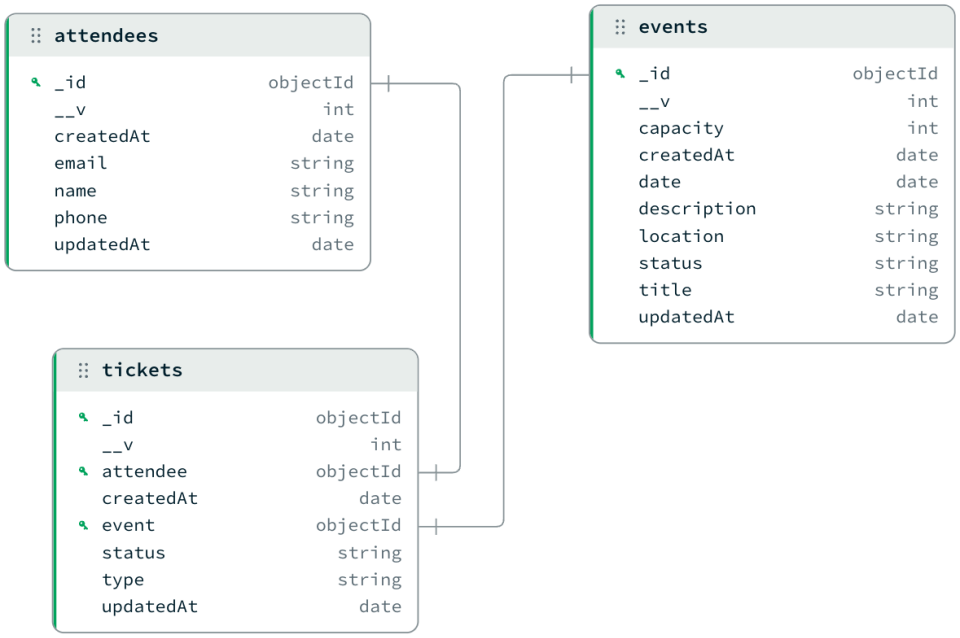

Figure 1: Entity Relationship Diagram

3 API Endpoints

3.1 POST /api/events

Creates a new event. `title`, `date`, and `status` are required. `status` must be one of `upcoming`, `ongoing`, `completed`, or `cancelled`. When an event is updated to `cancelled`, all its active tickets are automatically cancelled via a bulk update.

Request body:

```
{
  "title": "Tech Conference 2025",
  "description": "Annual developer meetup",
  "date": "2025-09-15T09:00:00.000Z",
  "location": "Malmo",
  "capacity": 200,
  "status": "upcoming"
}
```

Response 201:

```
{
  "success": true,
  "data": {
    "_id": "664a1f...",
    "title": "Tech Conference 2025",
    "description": "Annual developer meetup",
    "date": "2025-09-15T09:00:00.000Z",
    "location": "Malmo",
    "capacity": 200,
    "status": "upcoming",
    "createdAt": "2025-04-25T10:00:00.000Z",
    "updatedAt": "2025-04-25T10:00:00.000Z"
  }
}
```

3.2 POST /api/tickets

Registers an attendee to an event by creating a ticket. Both `event` and `attendee` must be valid ObjectIds referencing existing documents — the endpoint verifies both before inserting. `type` must be `standard` or `vip`.

Request body:

```
{
  "event": "664a1f...",
  "attendee": "664b2e...",
  "type": "standard"
}
```

Response 201:

```
{
  "success": true,
  "data": {
    "_id": "664c3d...",
    "event": "664a1f...",
    "attendee": "664b2e...",
    "type": "standard",
    "status": "active",
```

```
    "createdAt": "2025-04-25T10:05:00.000Z",
    "updatedAt": "2025-04-25T10:05:00.000Z"
  }
}
```

4 Challenges

Designing an effective central store was one of the more complex parts of the frontend. Getting the Context API to avoid unnecessary re-renders across collections required careful use of `useMemo` per collection, so that updating events, for example, would not re-render components that only consume attendees.

A time-consuming deployment issue turned out to be a forgotten step: the MongoDB Atlas cluster had IP access restricted to a specific address, and the Vercel deployment IP was not whitelisted. The API kept timing out with no clear error until the network access list in the Atlas dashboard was updated to allow connections from all IPs.